SIMATS ENGINEERING

ASSESSMENT TOOL 2 : Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2)

Question Count: 5 **Total Marks: 25**

Code of Conduct

I **B.V.THARUN (192525155)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student

Assessment Tasks Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering.
   - Design a cloud architecture using scalability and caching techniques.
   - Explain how load balancing and auto-scaling can ensure smooth streaming.
   - Discuss cost optimization strategies for such high-demand workloads.
2. A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments.
   - Propose a solution using container orchestration and multi-cloud strategies.
   - Explain how service discovery and scaling can be handled efficiently.
   - Highlight the benefits and risks of a multi-cloud deployment model.
3. An e-learning platform experiences data loss due to accidental deletion in the cloud. The organization needs a robust backup and disaster recovery strategy.
   - Design a solution using cloud-native backup, replication, and versioning.
   - Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved.
   - Provide steps to ensure business continuity during failures.
4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data. Some applications must remain on-premises due to compliance requirements.
   - Design a hybrid architecture that ensures secure communication between environments.
   - Explain how identity management and encryption should be implemented.
   - Discuss challenges in managing hybrid cloud environments.
5. A SaaS provider needs to ensure high availability and fault tolerance for its application. The system must continue operating even if one region goes down.
   - Propose a multi-region deployment strategy in the cloud.
   - Explain how data replication and failover mechanisms should be configured. ● Discuss monitoring and alerting strategies for proactive issue detection.

1. A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering. Design a cloud architecture using scalability and caching techniques. Explain how load balancing and auto-scaling can

ensure smooth streaming. Discuss cost optimization strategies for such high-demand workloads.

A) Scenario understanding and key issues: A video streaming platform has a very large and spiky audience, because demand rises sharply during new releases, live sports and popular series premieres. The content itself is heavy, since a single title is stored in many resolutions and bitrates, and viewers expect playback to start within a couple of seconds without interruption. The main risks are **buffering** caused by slow or distant delivery, **origin overload** when millions of users request the same content at the same moment, **under-provisioning** that makes the service crash at peak and **over-provisioning** that wastes money in quiet hours, and **very high bandwidth (egress) cost**, which is usually the biggest item in the bill. The design therefore needs horizontal scalability, multi-layer caching, global traffic routing and elastic capacity that follows demand automatically.

1. Proposed cloud architecture (scalability and caching):

The architecture begins with media ingestion and processing. Uploaded master videos are stored in object storage, and each upload creates an event in a message queue. A pool of transcoding workers reads the queue and converts every title into several bitrates and resolutions, then packages the output as HLS or MPEG-DASH segments of a few seconds each. This is called adaptive bitrate streaming, and it allows the player to change quality on the fly. Because the transcoding workers are stateless and queue-driven, the pool can grow or shrink with the number of uploads and is well suited to low-cost spot instances.

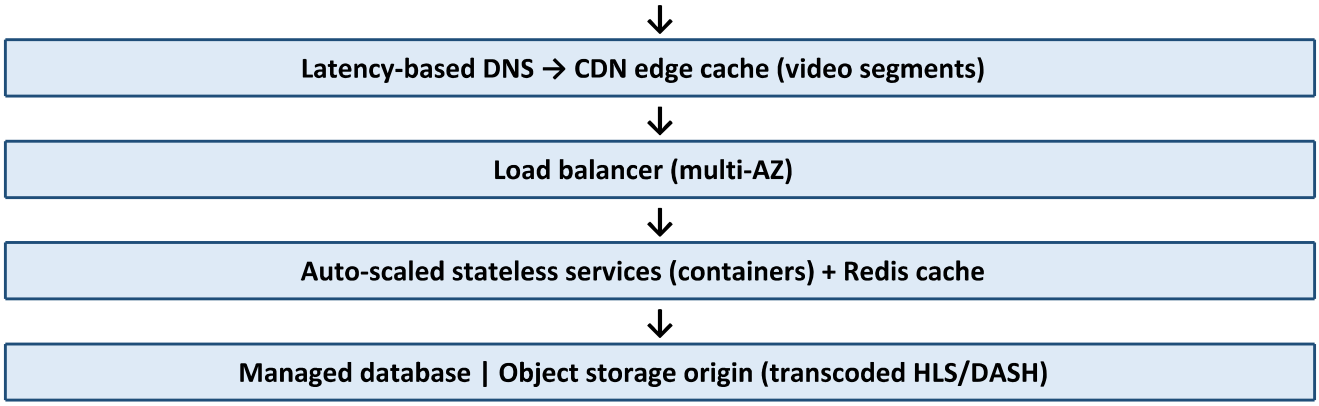
The processed renditions are kept in durable object storage such as Amazon S3, Azure Blob Storage or Google Cloud Storage, which acts as the origin. Object storage scales without a fixed limit, is replicated across availability zones, and is cheap per gigabyte. Instead of letting viewers read from the origin directly, a global Content Delivery Network (CDN) is placed in front of it. Popular segments are cached at edge locations that are physically close to viewers, so the majority of requests are answered from the edge and never reach the origin. An origin shield or regional cache is added as a middle layer so that, when an edge location misses, the request is collapsed with other similar requests and only one fetch goes to the origin. This is the most important scalability and caching technique in the whole design, because it protects the origin and reduces the distance the data travels.

The application tier handles everything other than the video bytes, such as login, subscription checks, the catalogue, search, recommendations and playback authorization. It is built as stateless microservices that run in containers on a managed Kubernetes service or a serverless container platform and are deployed in at least two or three availability zones behind load balancers. Because the services keep no session data inside the server, any instance can serve any request, which makes horizontal scaling simple. User profiles, watch history and catalogue data are stored in a managed NoSQL or distributed relational database that can be partitioned and replicated. In front of the database an in-memory cache such as Redis or Memcached keeps hot data like session tokens, watch lists, home-page rows and metadata, which reduces database load and gives sub-millisecond reads.

Traffic enters the system through latency-based DNS or a global accelerator that sends each viewer to the closest healthy region. Security is built in at the edge with TLS encryption, a web application firewall and DDoS protection, while the content itself is protected with Digital Rights Management (DRM) and short-lived signed URLs or tokens so that links cannot be shared freely. Finally, real-time monitoring collects metrics such as video start-up time, rebuffering ratio, average bitrate and error rate. These streaming-quality metrics are fed into alarms and scaling policies, so the platform reacts to what the viewer actually experiences and not only to server CPU.

Flowchart: Video streaming architecture





2. How load balancing and auto-scaling ensure smooth streaming:

Load balancing distributes incoming requests across many servers and availability zones so that no single machine becomes a bottleneck. Algorithms such as round-robin or least-connections spread the work evenly, and continuous health checks remove an unhealthy server from rotation within seconds, so users are not sent to a failed instance. Geo or latency-based routing sends each user to the nearest or healthiest region, and TLS offloading at the load balancer lowers the processing load on application servers. Since start-up time and buffering depend heavily on how quickly the first requests are served, an evenly balanced and healthy fleet keeps both low.

Auto-scaling adds capacity when demand rises and removes it when demand falls. Instances or containers are added when metrics such as CPU utilization, number of concurrent connections or requests per second cross a target value, which is known as target-tracking scaling. For events that are known in advance, such as a big premiere or a live match, scheduled scaling prepares the capacity before the audience arrives. Predictive scaling goes one step further and uses historic traffic patterns to forecast demand and scale ahead of it. Because a new instance takes time to boot and warm up, warm pools or pre-scaled capacity are kept ready so that extra servers join the fleet within seconds instead of minutes. When the peak is over, scale-in rules terminate the extra instances so the company stops paying for them.

Adaptive bitrate streaming works together with these two mechanisms on the client side. The player continuously measures the available bandwidth and buffer level and switches between the renditions that were prepared during transcoding. If a viewer's network becomes slow, the player moves to a lower resolution and playback continues without stopping, instead of freezing to wait for a high-quality segment. Finally, resilience is added by deploying across multiple availability zones and regions and by using more than one CDN, so that a failure of one zone, region or CDN provider only shifts traffic and does not interrupt the stream. In this way load balancing, auto-scaling, adaptive bitrate and redundancy each remove a different cause of buffering, and together they keep the experience smooth even at very high concurrency.

Table: Load balancing and auto-scaling techniques

Technique	How it helps
Load balancing	Distributes requests across many servers and zones using round-robin / least-connections; health checks remove failed servers; DNS/geo routing sends users to the nearest or healthiest region; TLS offloading lowers server load. No single server becomes a bottleneck, so start-up time and buffering stay low.
Auto-scaling	Adds instances/containers when metrics such as CPU, concurrent connections or requests per second cross a threshold (target tracking) and removes them when demand falls. **Scheduled scaling** prepares for known peaks (a premiere), **predictive scaling** uses historic

	patterns, and **warm pools / pre-scaled capacity** avoids slow start-up.
Adaptive bitrate (ABR)	The player switches quality according to bandwidth, so playback continues without stopping even on slow networks.
Resilience	Multi-AZ/multi-region deployment and multi-CDN failover keep the stream running during failures.

3. Cost optimization strategies:

Because streaming workloads move enormous amounts of data, cost optimization begins with the CDN. Maximizing the cache-hit ratio is the single most effective step, and it is achieved by giving immutable video segments a long time-to-live, using consistent cache keys, and enabling an origin shield. A higher hit ratio means fewer requests to the origin, less origin compute and less data transferred from the cloud, which is normally the largest cost. Negotiating committed-use pricing with the CDN, using more than one CDN to obtain better bandwidth rates, and avoiding unnecessary cross-region data transfers also reduce the bill.

Storage and encoding choices are the second area. Storage tiering and lifecycle rules keep new and popular titles in hot storage and automatically move older, rarely watched content to infrequent-access or archive tiers, which are much cheaper per gigabyte. Efficient encoding, such as per-title encoding and modern codecs like H.265 or AV1, delivers the same visual quality with fewer bytes, so every stream costs less to deliver. For compute, a mixed purchasing model is used: reserved instances or savings plans cover the steady baseline load, on-demand instances absorb the peaks, and low-cost spot instances run the fault-tolerant transcoding jobs, which can safely be interrupted and retried. Right-sizing instance types and scaling in aggressively remove idle servers.

Lastly, cost governance makes savings continuous instead of one-time. Resources are tagged by team and workload, budgets and alerts warn when spending crosses a threshold, and regular cost reports are reviewed to find unused volumes, oversized instances and unexpected data-transfer charges. Financial operations practices of this kind ensure that the platform scales its performance for millions of users without scaling its cost at the same rate.

Justification and conclusion: CDN caching moves content close to users, which gives low latency and low origin load, while stateless services, load balancing and auto-scaling match capacity to demand in real time. Adaptive bitrate streaming and multi-zone, multi-CDN redundancy protect the viewing experience during network problems and failures, and storage tiering, efficient encoding and a mixed purchasing model keep the cost under control. Together these techniques deliver minimal buffering for millions of concurrent users at an optimized cost, which is exactly what the media company requires from its cloud migration.

2. A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments. Propose a solution using container orchestration and multi-cloud strategies. Explain how service discovery and scaling can be handled efficiently. Highlight the benefits and risks of a multi-cloud deployment model.

A) Scenario understanding and key issues: The fintech startup runs its containerized applications on more than one cloud provider. Every provider has different APIs, networking models, identity systems, load balancers and management tools, and when each environment is built and changed by hand the result is **configuration drift**, inconsistent environments, slow and error-prone deployments, and poor visibility into what is running where. Because the business is financial, security and compliance obligations such as PCI-DSS, strong audit trails and data protection must be identical in every environment, and the team must be

able to release changes quickly without risking an outage. The solution therefore needs one standard way to package, deploy, secure and observe applications that works the same on every cloud.

1. Proposed solution (container orchestration + multi-cloud strategy):

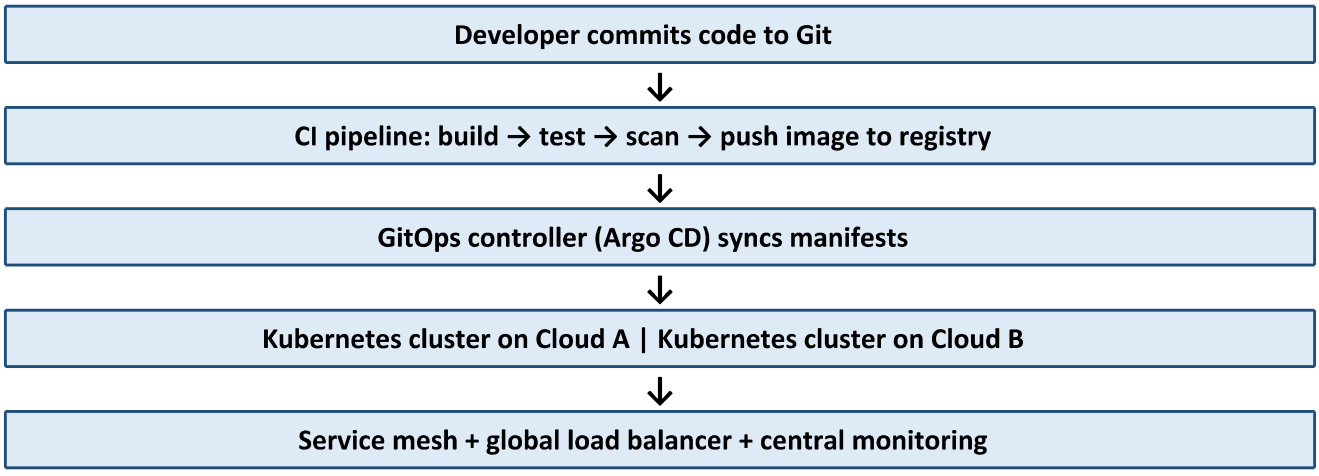
The first step is to standardize packaging. Every service is built once as a Docker or OCI container image and pushed to a common registry that performs vulnerability scanning and image signing. Because the same immutable artifact is promoted from testing to production and runs unchanged on every cloud, the "works on my machine" and "works only on one provider" problems disappear. Configuration that differs between environments, such as endpoints and credentials, is injected at deployment time through configuration maps and a secrets manager and is never baked into the image.

Kubernetes is then used as the common control plane. The startup runs managed Kubernetes services such as Amazon EKS, Azure AKS and Google GKE with the same version and the same baseline configuration, or it adopts a multi-cluster platform such as Rancher, OpenShift or Anthos to manage all clusters from one place. Kubernetes hides most provider differences behind the same objects, namely pods, deployments, services, ingress and persistent volume claims, so an application definition written once can be applied to any cluster. This portability is the heart of the multi-cloud strategy, because workloads can be moved or duplicated between providers without being rewritten.

Consistency is enforced with Infrastructure as Code and GitOps. Terraform describes the clusters, networks, identity roles and managed services of every cloud, while Helm charts or Kustomize overlays describe the applications. All of this is stored in version control, so every environment can be recreated from code and every change is reviewed, tested and traceable. A CI/CD pipeline builds the image, runs unit and security tests, scans for vulnerabilities and pushes the image to the registry. A GitOps controller such as Argo CD or Flux then continuously compares the desired state in Git with the actual state of each cluster and synchronizes them. Drift is corrected automatically, rollbacks are as simple as reverting a commit, and rolling, blue-green or canary releases can be used to reduce risk.

Security and policy are applied in the same uniform manner. Secrets are kept in a central vault such as HashiCorp Vault instead of in code, policy-as-code tools such as Open Policy Agent and Gatekeeper reject non-compliant deployments, role-based access control and network policies limit what each workload can do, and a service mesh provides mutual TLS between services. Observability is centralized with Prometheus and Grafana for metrics and a central logging platform such as ELK or Loki, giving the operations team one view of all clusters. Finally, a DNS service or global load balancer routes users to the nearest healthy cluster and allows traffic to be moved to the other cloud during an incident.

Flowchart: Multi-cloud deployment flow



2. Service discovery and scaling:

Inside a single cluster, service discovery is handled by Kubernetes itself. Each Service object gets a stable DNS name and virtual IP through CoreDNS, and the pods behind it are registered and removed automatically as they start, fail or are rescheduled, so applications never need hard-coded addresses. Across clusters and clouds the same idea is extended with a service mesh such as Istio or Linkerd, or with Consul using multi-cluster federation, together with ingress controllers and global DNS. Services can then find and call each other by name regardless of which provider hosts them, and the mesh adds load balancing, retries, timeouts and encrypted traffic between them.

Scaling is handled at several levels. The Horizontal Pod Autoscaler adds or removes pods according to CPU, memory or custom metrics such as requests per second, and the Cluster Autoscaler adds or removes worker nodes when pods cannot be scheduled or nodes are underused. The Vertical Pod Autoscaler helps right-size the resource requests of each container, and event-driven tools such as KEDA scale workloads on queue length or other external events. Setting resource requests and limits for every container prevents noisy-neighbour problems, so one service cannot starve another.

Efficient deployment management completes the picture. Rolling updates replace pods gradually, readiness and liveness probes make sure traffic is sent only to healthy pods, and blue-green and canary strategies allow a new version to be tested on a small share of users before full release. If health checks fail, the deployment is rolled back automatically. These mechanisms reduce downtime and make frequent releases safe, which is important for a startup that needs to ship features quickly.

Table: Service discovery and scaling approaches

Area	Efficient approach
Service discovery	Inside a cluster, Kubernetes Services and CoreDNS give each service a stable name and virtual IP, and pods are registered automatically as they start/stop. Across clusters/clouds, use a service mesh (Istio/Linkerd) or Consul with multi-cluster federation, plus ingress controllers and global DNS, so services find each other by name regardless of provider.
Scaling	**Horizontal Pod Autoscaler** adds/removes pods based on CPU, memory or custom metrics; **Cluster Autoscaler** adds/removes worker nodes; **VPA** right-sizes resource requests; **KEDA** scales on events/queue length. Resource requests and limits prevent noisy neighbours.
Deployment management	Rolling updates, blue-green and canary releases with readiness probes reduce downtime; automatic rollback on failed health checks.

3. Benefits and risks of multi-cloud deployment:

A multi-cloud model offers several important benefits. It avoids vendor lock-in and improves the startup's bargaining power on price and terms, and it increases availability and resilience because the application can survive the outage of an entire provider. The company can use the best service from each provider, place workloads in the regions that satisfy data-residency and regulatory requirements, and optimize cost and performance by running each workload where it is cheapest or fastest. A second provider also gives a natural option for disaster recovery that is independent of the first.

At the same time the model carries real risks. The design and operation become considerably more complex, and the team needs engineers who understand several platforms, which creates a skills gap. The attack surface is larger, and identity and security settings can become inconsistent if they are managed separately in each cloud. Data egress charges and inter-cloud latency can increase cost and slow down applications that talk across providers, and monitoring, data consistency and compliance auditing become more fragmented. These risks are reduced by using common tooling, one identity and policy framework, central observability and strong automation, so that the organization manages one platform instead of several.

Table: Benefits and risks of multi-cloud deployment

Benefits	Risks
Avoids vendor lock-in and improves bargaining power	Higher complexity of design and operations
Higher availability and resilience (survives a provider outage)	Skills gap; need engineers who know several platforms
Use best-of-breed services and meet data-residency/regulatory needs	Larger attack surface and inconsistent IAM/security settings
Optimize cost and performance by placing workloads where best	Data egress charges and inter-cloud latency
Disaster recovery across providers	Fragmented monitoring, data consistency and compliance auditing

Justification and conclusion: Kubernetes combined with GitOps and Infrastructure as Code gives the startup the same version-controlled environment on every cloud, which directly solves the consistency and deployment problems described in the scenario. Service meshes and autoscaling make discovery and capacity management efficient, and common security policy, central monitoring and automation keep the risks of multi-cloud under control while delivering resilience, flexibility and compliance for the fintech business.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud. The organization needs a robust backup and disaster recovery strategy. Design a solution using cloud-native backup, replication, and versioning. Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved. Provide steps to ensure business continuity during failures.

A) Scenario understanding and key issues: In this scenario, important data of the e-learning platform, such as course content, student records and assignment submissions, was permanently deleted by mistake. The root causes are the **lack of versioning and backups**, delete permissions that were too broad, no copy of the data in a second location, and the absence of a tested recovery plan. A cloud provider keeps the infrastructure available, but it does not protect the customer from the customer's own mistakes, because under the shared responsibility model protecting and recovering data is the customer's job. The new solution must therefore make deletion reversible, keep independent copies of the data, and make recovery fast and predictable.

1. Solution design (cloud-native backup, replication and versioning):

The first layer of protection is versioning and soft delete. Object versioning is enabled on all storage buckets, so that when a file is deleted or overwritten the older version is retained and can be restored in seconds. Soft-delete or recycle-bin features keep deleted items for a retention period, and for critical data the organization adds MFA-delete or Object Lock so that a version cannot be removed immediately even by an authorized user. This layer alone would have prevented the permanent loss described in the scenario, because a mistaken deletion becomes a simple restore operation.

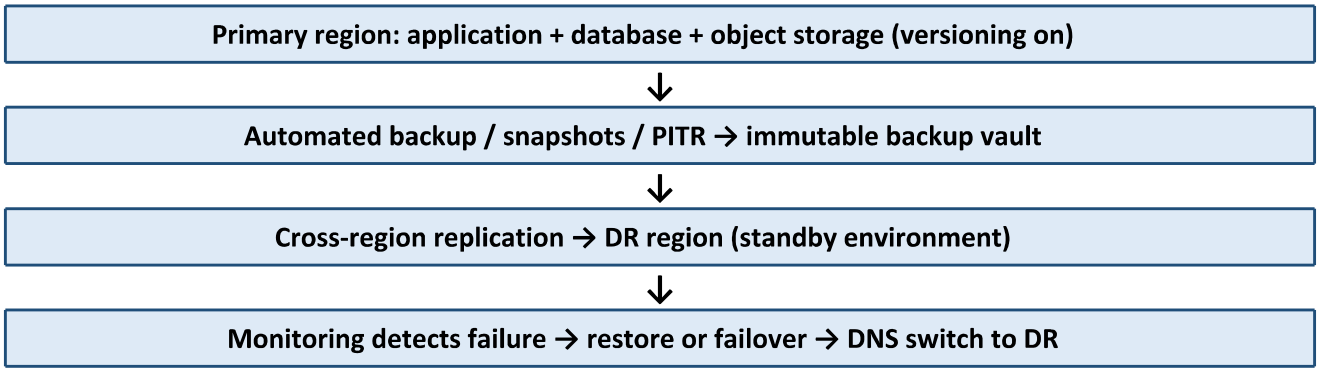
The second layer is automated backup. A managed backup service such as AWS Backup, Azure Backup or Google Backup and DR is configured with policies that take daily full backups and frequent incremental backups of databases, block volumes and file shares. Snapshots are scheduled automatically, and databases are configured with point-in-time recovery (PITR), which keeps transaction logs so that the database can be rolled back to any second within the retention window, for example to just before the

faulty delete command. Because backup is policy-based and centrally managed, nothing depends on a person remembering to run it.

The third layer is replication. Object storage is replicated to a second region using cross-region replication, and the database is given a cross-region read replica or standby instance. This means that the data also exists in a physically separate location, which protects against the loss of a whole zone or region and gives a starting point for disaster recovery. Replication alone is not a backup, since a deletion is also replicated, which is why it is always combined with versioning and point-in-time backups.

The fourth layer is protection of the backups themselves. Backups are placed in a separate account or vault with vault-lock or WORM (write once, read many) settings, so that they are immutable and cannot be deleted by mistake, by a compromised administrator account or by ransomware. This is supported by least-privilege access control, in which IAM roles limit who can delete data, an approval workflow is required for destructive actions, and every action is recorded in audit logs such as CloudTrail or the Azure activity log. Retention and lifecycle policies keep, for example, daily backups for thirty days and monthly backups for one year, and move older backups to cheaper archive tiers to control cost. Finally, the whole environment is described as Infrastructure as Code, so the networks, servers and services can be recreated quickly in another region when needed.

Flowchart: Backup and disaster recovery flow



2. How RTO and RPO are achieved:

The Recovery Point Objective (RPO) is the maximum amount of data loss the business can accept, measured in time. It is achieved by how often data is copied or replicated. Frequent incremental backups, continuous point-in-time recovery, near-synchronous replication and object versioning all reduce the gap between the last good copy of the data and the moment of failure, so a lower RPO is obtained by copying more often. The Recovery Time Objective (RTO) is the maximum acceptable downtime, and it is achieved by how quickly a working system can be brought back. Pre-built standby infrastructure, automated failover, Infrastructure as Code templates, tested runbooks and fast restore storage all shorten the time between the failure and the point at which users can work again.

Several disaster recovery strategies offer different trade-offs between cost and recovery speed. In the backup-and-restore approach, backups are restored only when needed, so the RPO and RTO are measured in hours, but it is the cheapest option. In a pilot-light approach the data is continuously replicated and only the minimal core services are kept ready, which gives an RPO of minutes and an RTO of tens of minutes to a few hours, because the rest of the environment is scaled up on failure. In a warm-standby approach a scaled-down but fully working copy of the system always runs in the second region, which gives an RPO of seconds to minutes and an RTO of minutes, since only scaling up and a DNS switch are required. In a multi-site active-active approach full capacity runs in two regions at the same time, giving near-zero RPO and RTO at the highest cost.

Table: Disaster recovery strategies (RPO / RTO)

DR strategy	Typical RPO	Typical RTO	Description
Backup and restore	Hours	Hours	Backups restored when needed; lowest cost, slowest recovery
Pilot light	Minutes	Tens of minutes to hours	Data replicated, minimal core services kept ready and scaled up on failure
Warm standby	Seconds to minutes	Minutes	Scaled-down full copy always running; scale up and switch DNS
Multi-site active-active	Near zero	Near zero	Full capacity in two regions; highest cost

Recommendation: For an e-learning platform, warm standby for the database and application, together with versioning and point-in-time recovery for files, is a balanced choice. It can meet a target such as an RPO of fifteen minutes or less and an RTO of about one hour, although the final targets must be agreed with the business, and it does so at a reasonable cost. Accidental deletions, which were the cause of the original incident, are then repaired within minutes by restoring a previous version, without invoking a full failover.

3. Steps to ensure business continuity during failures:

Business continuity is achieved by following a clear and rehearsed sequence of actions. The first step is detection: monitoring and alarms identify data loss, an outage or a replication failure as early as possible. The second step is to declare the incident and communicate, which means that the incident team activates the disaster recovery plan and informs users and management so that everyone knows the situation and expected timeline. The third step is containment, in which the cause is stopped, for example by revoking the faulty permission or disabling the faulty script, so that no further data is lost while recovery is in progress.

The fourth step is recovery. Deleted objects are restored from earlier versions, the database is restored to a point just before the error using PITR, or the application is failed over to the standby region if the primary region is unavailable. In the fifth step, traffic is redirected by switching DNS or the global load balancer to the recovered or disaster recovery environment. In the sixth step, the team validates the result by running integrity and functional checks and confirming that students and teachers can open courses and submit work. Once the primary region is stable again, the seventh step is to fail back in a controlled way and to hold a post-incident review that records the cause, the timeline and the improvements needed. The last and ongoing step is regular testing, because scheduled disaster recovery drills, restore tests and updated runbooks prove that the RTO and RPO targets are real and not assumed.

Justification and conclusion: Versioning and soft delete protect against accidental deletion, backups and point-in-time recovery protect against corruption and mistakes, and cross-region replication protects against the loss of a system or region. Immutable storage and least-privilege access prevent the same failure from happening again, and a tested disaster recovery plan guarantees the agreed RTO and RPO. Together they keep courses and student data available and protected, so the platform can continue its educational service even during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data. Some applications must remain on-premises due to compliance requirements. Design a hybrid architecture that ensures secure communication between environments. Explain

how identity management and encryption should be implemented. Discuss challenges in managing hybrid cloud environments.

A) Scenario understanding and key issues: The government agency handles two kinds of data and applications. Sensitive data and compliance-bound applications must remain in the agency's own data center because of legal and regulatory rules, while non-sensitive workloads, such as public information portals and analytics on anonymized data, can use the public cloud to gain elasticity and lower cost. A hybrid cloud is the right model, but it brings its own key risks, namely **data leakage** between the environments, **insecure connectivity** between the data center and the cloud, **inconsistent identity and policy** across two platforms, and the possibility of **failing a compliance audit** if controls cannot be demonstrated. The architecture must therefore keep control of sensitive assets while still allowing safe and efficient integration with the cloud.

1. Hybrid architecture with secure communication:

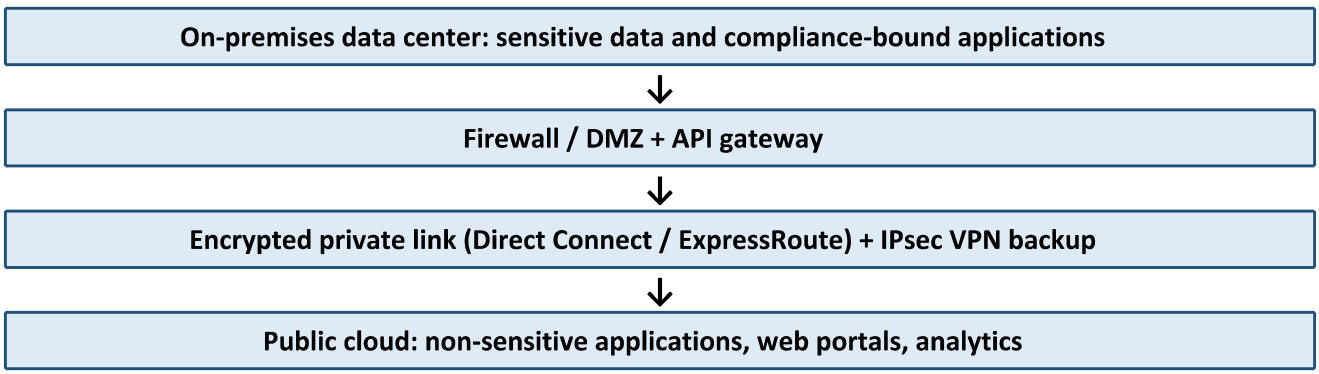
The design starts with data classification. All data is classified into categories such as public, internal and sensitive, and the classification decides where each workload may run. Sensitive data and regulated systems remain in the on-premises private cloud, which gives the agency physical control, while public-facing and non-sensitive workloads run in the public cloud. Making this decision at the beginning, and recording it in policy, prevents sensitive information from drifting into the wrong environment later.

Secure communication between the two environments is provided by private connectivity. A dedicated private link such as AWS Direct Connect or Azure ExpressRoute carries traffic over a dedicated path instead of the public internet, which gives predictable bandwidth and latency and reduces exposure. An IPsec site-to-site VPN is configured as an encrypted backup path, so that connectivity continues if the dedicated link fails. All traffic over these links is encrypted, and the connection is terminated on firewalls that inspect and filter it.

Inside both environments, the network is segmented. A hub-and-spoke topology is used with separate virtual networks and subnets for different applications, a demilitarized zone (DMZ) is placed at the boundary, and next-generation firewalls, security groups and micro-segmentation restrict traffic to what each application actually needs. Internal systems are never exposed directly to the internet. Integration between the two sides is controlled through an API gateway or integration layer, which is the only permitted path for data exchange. Before any record is sent to the cloud it is tokenized, masked or anonymized, so that even if the cloud side is compromised the real sensitive values are not exposed.

A unified management layer such as Azure Arc, Google Anthos or vCenter integration gives the agency a single place to apply policy, patching and monitoring across both environments. Underlying the whole design is the zero-trust principle: no request is trusted merely because it comes from inside the network, and every request is authenticated, authorized and logged regardless of its location.

Flowchart: Hybrid cloud architecture



2. Identity management and encryption:

For identity management, the agency should maintain one authoritative identity source. The on-premises Active Directory is synchronized with a cloud identity provider such as Microsoft Entra ID, and federation using SAML 2.0 or OpenID Connect gives users Single Sign-On across both environments, so each person has a single identity and a single set of credentials to manage. Multi-factor authentication is enforced for all users, access is granted using role-based access control with the principle of least privilege, and conditional access rules take into account the user's device, location and risk level. Administrators use privileged access management with just-in-time elevation, tokens are short-lived, and all authentication and authorization events are written to central audit logs. Joiners, movers and leavers are handled automatically, so that access is removed promptly when a person changes role or leaves.

Encryption must protect data in transit, at rest and during key handling. For data in transit, TLS 1.2 or 1.3 is used for all application traffic, IPsec protects the site-to-site tunnels, MACsec can optionally be added on dedicated links, and mutual TLS is used between services so that both sides prove their identity. For data at rest, AES-256 encryption is applied to disks, databases, object storage and backups in both environments. Key management is critical in a government setting. A central Key Management Service or hardware security module (HSM) is used, customer-managed keys are preferred, and keys protecting the most sensitive data are generated and held on-premises (bring-your-own-key or hold-your-own-key models), so the cloud provider cannot read that data without the agency's cooperation. Keys are rotated regularly, duties are separated so that no single administrator controls everything, and every use of a key is logged.

Beyond encryption, additional data-protection controls are used. Sensitive fields are tokenized or masked before leaving the data center, data loss prevention (DLP) rules detect and block attempts to move sensitive information to unauthorized locations, and backups are encrypted with the same care as production data. Together, strong identity and strong encryption ensure that only the right people and systems can access the data, and that the data remains unreadable if it is intercepted or stolen.

Table: Identity management and encryption

Area	Implementation
Identity and access management	One authoritative identity source: on-premises Active Directory synchronized with a cloud identity provider (e.g. Microsoft Entra ID) using federation (SAML 2.0 / OpenID Connect) for Single Sign-On. Enforce multi-factor authentication, role-based access control with least privilege, conditional access, privileged access management for administrators, short-lived tokens, and central audit logs.
Encryption in transit	TLS 1.2/1.3 for all application traffic, IPsec for site-to-site tunnels, optionally MACsec on dedicated links; mutual TLS between services.
Encryption at rest	AES-256 encryption of disks, databases and object storage in both environments.
Key management	Central Key Management Service/HSM; customer-managed keys (BYOK/HYOK) with keys for sensitive data held on-premises; key rotation, separation of duties and access logging.
Data protection	Tokenization or masking of sensitive fields, data loss prevention (DLP) rules and backup encryption.

3. Challenges in managing hybrid cloud:

Managing a hybrid environment is considerably harder than managing a single one. The first challenge is integration and interoperability, because legacy on-premises systems were not designed for cloud services and often use different protocols, data formats and release cycles. The second challenge is applying consistent security and compliance policy across two different environments and then proving it to

auditors, since each environment has its own tools, logs and configuration models. Closely related is the challenge of visibility and monitoring, as the tools that watch the data center often differ from those that watch the cloud, which makes it difficult to see the whole system and to find the cause of a problem quickly.

Network performance is another concern. Latency, bandwidth limits and the availability of the link between the sites affect applications that depend on data from both sides, so architects must decide carefully which components should be placed together. Data governance and synchronization are also demanding, because the agency must keep data consistent across environments and prevent sensitive information from moving to the wrong place through misconfiguration or human error. Operationally, the organization faces a skills shortage, since staff must understand both traditional infrastructure and cloud platforms, and it must control tool sprawl and cost, since two environments plus data-transfer charges can quickly become expensive. Disaster recovery and change management must also span both environments, so a failure or an update in one place does not break the other. These challenges are reduced with unified management platforms, infrastructure as code, policy-as-code, common identity, centralized logging and well-defined data governance.

Justification and conclusion: Keeping regulated workloads on-premises while using private and encrypted connectivity, federated identity and customer-controlled keys gives the agency the flexibility and cost benefits of the cloud without giving up compliance or control. Network segmentation, an API-gated integration layer and zero-trust principles limit the impact of any single failure, while unified management and policy-as-code reduce the operational difficulties of running a hybrid environment. This design therefore satisfies the agency's security, compliance and efficiency requirements.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application. The system must continue operating even if one region goes down. Propose a multi-region deployment strategy in the cloud. Explain how data replication and failover mechanisms should be configured. Discuss monitoring and alerting strategies for proactive issue detection.

A) Scenario understanding and key issues: The SaaS provider's customers expect the application to be available at all times, so the failure of an entire cloud region must not stop the service. The main risks are a **single point of failure**, such as one region, one database or one load balancer, **data loss or inconsistency** during failover, **slow or manual failover** that extends downtime, and **late detection** of problems that lets small faults grow into outages. High availability means the system is designed to minimize downtime, and fault tolerance means it continues to operate correctly even when components fail. The design therefore needs redundancy at every layer, automated failover and strong monitoring that warns the team before users are affected.

1. Multi-region deployment strategy:

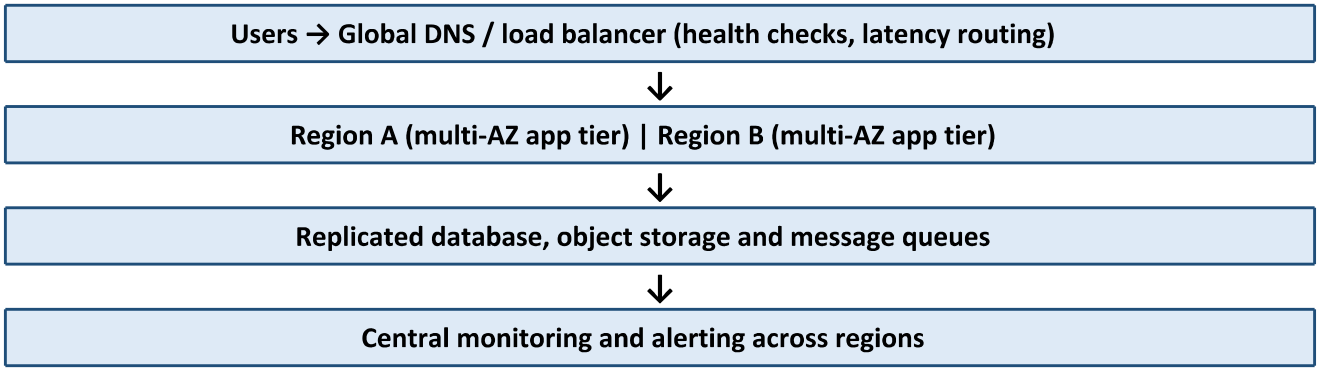
The foundation is to deploy the application in two or more regions, with each region itself spread across multiple availability zones. The same stack is deployed in every region using Infrastructure as Code, so the environments are identical and can be rebuilt quickly. Within a region, the use of several availability zones means that the failure of a single data center does not even require a regional failover, while the second region protects the service from a complete regional outage caused by a natural disaster, power failure or major software fault at the provider.

The provider must then choose between an active-active and an active-passive model. In an active-active design, both regions serve users at the same time, which gives lower latency because users are routed to the nearest region, avoids cold starts and uses the paid capacity fully, but it requires careful data replication and conflict handling. In an active-passive design, the second region is a warm standby that is

kept ready but does not serve normal traffic, which costs less and is simpler to manage but fails over more slowly. Active-active is preferred for a SaaS product with global users and strict availability goals, while active-passive is an acceptable and cheaper starting point.

Global traffic management directs users to the correct region. Latency-based or geo-based DNS, or a global load balancer such as Amazon Route 53, Azure Traffic Manager or AWS Global Accelerator, routes each user to the nearest healthy region and uses continuous health checks to stop sending traffic to a region that fails. The application tier is kept stateless, with containers or virtual machines behind regional load balancers and auto-scaling groups, and user sessions are stored in a distributed cache or encoded in tokens so that a user can be served by any instance in any region. Each region is also given capacity headroom, which means it is sized, or can scale quickly enough, to handle one hundred percent of the traffic if the other region fails. A CDN in front of the application serves static assets and cached content, which reduces load and keeps parts of the service working even during an incident.

Flowchart: Multi-region design



2. Data replication and failover configuration:

Data is the hardest part of a multi-region design, because the application can be moved quickly but data must be kept consistent. For the database, a managed multi-region or global database such as Amazon Aurora Global Database, DynamoDB global tables, Azure Cosmos DB or Google Cloud Spanner is used. Asynchronous replication gives low write latency and a recovery point of only a few seconds, which is acceptable for most workloads, whereas synchronous replication can be chosen for the most critical data to obtain zero data loss, at the cost of higher latency. If more than one region accepts writes, conflict-resolution rules such as last-writer-wins or versioning must be defined, and in an active-passive setup the promotion of the replica to the primary database should be automated and rehearsed.

Object storage and files are protected with cross-region replication and versioning so that uploaded content is available in both regions. Caches are treated as rebuildable and are refilled from the database after a failover, whereas message queues and event streams are replicated or mirrored, and consumers are designed to be idempotent so that a message processed twice during failover does not create duplicate effects.

The failover mechanism itself must be automatic and fast. Health checks, for example every ten seconds with a failure declared after three consecutive failed checks, trigger the DNS or global load balancer to remove the failed region from rotation, and a low DNS time-to-live of around sixty seconds makes clients pick up the change quickly. Inside the application, circuit breakers, timeouts and retries with back-off prevent a failing dependency from causing a cascade of failures, and automated runbooks perform tasks such as database promotion without waiting for manual intervention. After the failed region has been repaired, a controlled failback is performed once the data has been fully resynchronized. The entire design is validated through regular game days and chaos testing, in which a regional failure is simulated deliberately, so that the real recovery time and data loss are measured and proven instead of assumed.

Table: Data replication and failover configuration

Component	Configuration
Database	Use a managed global/multi-region database (Aurora Global Database, DynamoDB global tables, Cosmos DB, Cloud Spanner). Asynchronous replication gives low latency and an RPO of seconds; synchronous replication for the most critical data gives zero data loss at the cost of latency. For multi-writer setups define conflict-resolution rules (e.g. last-writer-wins or versioning). For active-passive, automate promotion of the replica to primary.
Object storage and files	Cross-region replication with versioning enabled.
Cache and queues	Rebuild caches from the database; replicate or mirror message queues/streams, with idempotent consumers to avoid duplicate processing.
Failover mechanism	Health checks (for example every 10 s, failing after 3 checks) trigger automatic DNS/global load balancer failover; use a low DNS TTL (about 60 s); circuit breakers and retries in the application; automated runbooks for database promotion.
Failback and testing	Controlled failback after resync; regular game days / chaos testing (simulating a region failure) to prove RTO and RPO.

3. Monitoring and alerting for proactive detection:

Monitoring should start with the four golden signals, which are latency, traffic, errors and saturation, measured per service and per region. These are supplemented with infrastructure indicators such as CPU, memory and connection usage and with a special focus on database replication lag, because a growing lag directly increases the amount of data that could be lost in a failover. Centralized logging and distributed tracing help engineers follow a single request across many services and find quickly which component is failing or slow.

Synthetic monitoring complements real-user data. Scripted transactions and health probes are run continuously from several locations, which allows the team to detect that a login or checkout flow has stopped working even before customers report it. Alerting is based on service level objectives (SLOs), so alerts are raised when the error budget is being consumed too quickly, and multi-level thresholds with warning and critical states reduce noise and alert fatigue. Notifications are delivered through email, SMS, chat and on-call tools such as PagerDuty or Amazon SNS, with escalation policies and documented runbooks so that the right person responds in time. Where it is safe, auto-remediation actions such as restarting a failed instance, scaling out or triggering a failover are run automatically.

Finally, cross-region dashboards give a single view of system health, machine-learning-based anomaly detection highlights unusual trends such as slowly rising latency, and regular capacity reviews and post-incident reviews feed lessons back into the design. This continuous cycle of measurement, alerting and improvement allows problems to be found and fixed proactively, before they develop into outages.

Justification and conclusion: Redundant regions with replicated data, automated and health-checked failover, and proactive monitoring remove the single points of failure from the SaaS architecture. As a result, the application continues to operate even if an entire region goes down, with minimal data loss and minimal downtime, while regular testing and continuous monitoring ensure that the availability targets promised to customers are actually met.

Scenario-Based Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented

Total Marks Awarded: